Basic (design on paper before touching code) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `architecture_comparison_practice.py`: a prediction written *before* any code runs is the only honest way to find out later whether your design instinct was actually right. This section is kept as a plain-text comment block at the very top of the file, so the real measured numbers produced below it can be checked directly against what you predicted. **If not:** after building and measuring, you'd just remember "I expected that" — whether you actually did or not, with no written record to check yourself against.

Task chosen below: "research the top 3 causes of X, then write a 3-sentence summary."

Solution

Story: a vague guess becomes genuinely useful the moment it has specific numbers and named mechanisms a colleague could actually argue with and check. **If not:** a vague claim like "the supervisor is slower" can never be proven wrong — and if it can't be proven wrong, it can't teach you anything either.

```
design 1 -- single agent
    flow: one agent gets "research X, write a summary," calls a research tool as
    needed inside its own turn, then writes the final summary itself
    cost: ~1-2 LLM calls total
    speed: fastest of the three -- no agent-to-agent handoff at all
    reliability risk: the single system prompt has to be good at both research
    judgment and writing style simultaneously; a prompt tuned for good writing
    style can quietly get worse at judging source quality, and vice versa

design 2 -- sequential
    flow: research_agent(topic) -> findings; write_agent(findings) -> summary,
    always in that fixed order
    cost: ~2 LLM calls, one per stage
    speed: about the same wall-clock time as design 1 -- there's no parallelism
    to gain here, since writing genuinely needs the research first
    reliability risk: very predictable in structure (the order is enforced by
    code, not judgment), but there is no check step -- a weak research_agent
    output is faithfully summarized as if it were solid
```

```
design 3 -- supervisor
    flow: supervisor reads the task, decides to call research_agent, gets the
    result, decides to call write_agent, gets the result, returns
    cost: ~3+ LLM calls -- 2 for the actual work, 1 or more for the routing
    decision itself, which is not free
    speed: slowest of the three -- the routing decision adds real delay on
    top of design 2's delay, for a task whose order was fixed anyway
    reliability risk: everything design 2 has, plus a new failure mode --
    the supervisor could route to the wrong specialist, or call one specialist
    twice thinking it needs another pass
```